Formalizing $R(3,6) = 18$ in Lean 4: A Didactic Account

Zar Goertzel Oruži*

February 28, 2026

Abstract

We describe a complete, machine-verified proof of the Ramsey number $R(3,6) = 18$ in Lean 4 using Mathlib. The formalization follows Cariolaro's elementary proof for the upper bound and the Graver–Yackel graph for the lower bound. A key technical contribution is the *IndepSetChecker* architecture: a kernel-efficient backtracking independent-set search that avoids both the memory exhaustion of naive `decide` and the trust-expanding axiom `Lean.ofReduceBool` introduced by `native_decide`. The final theorem depends on exactly the three standard Lean 4 axioms `propext`, `Classical.choice`, and `Quot.sound`, with no `sorry` and no problem-specific axioms.

Contents

1 Introduction **3**

2 Proof Overview and Lean Setup **3**
2.1 Definitions . . . 3
2.2 The Main Theorem . . . 4
2.3 Module Structure . . . 4
2.4 Reviewer Reproducibility Checklist . . . 4

3 The Lower Bound: The Graver–Yackel Graph **5**
3.1 Definition in Lean . . . 5
3.2 The Lower Bound Argument . . . 6

4 The IndepSetChecker Architecture **6**
4.1 The Problem with Naive Decidability . . . 6
4.2 Four-File Architecture . . . 6
4.3 The Search Function . . . 7
4.4 Sub-Problem Splitting . . . 8
4.5 Structural Decomposition: Combining Sub-Problems . . . 9
4.6 Correctness: The Completeness Theorem . . . 9
4.7 Bridge to Mathlib's `SimpleGraph` . . . 10
4.8 Triangle-Freeness via the Complement . . . 11
4.9 Summary of the Architecture . . . 12

Oruži is a collaborative AI entity: ChatGPT 5., Claude Code 4.*, and Codex 5.*.

5 The Upper Bound: Formalizing Cariolaro's Proof **12**
5.1 Setup . . . 12
5.2 Claim 1: G is 5-Regular . . . 13
5.3 Claim 2: Neighbor Structure . . . 13
5.4 Claim 3: P Induces a 4-Cycle . . . 13
5.5 Final Contradiction . . . 14

6 Small Ramsey Numbers as Dependencies **14**

7 Equivalence with the Textbook Definition **15**
7.1 The Textbook Definition . . . 15
7.2 Instance Independence . . . 15
7.3 Equivalence Theorem . . . 15
7.4 Main Instantiation . . . 16

8 Axiom Footprint and Trust **16**

9 Conclusion **16**

1 Introduction

The Ramsey number $R(k, l)$ is the smallest positive integer n such that every 2-coloring of the edges of the complete graph K_n contains either a monochromatic k-clique or a monochromatic l-independent set. Equivalently, every graph on n vertices either contains a k-clique or an l-independent set. In everyday language: any gathering of $R(3, 6) = 18$ people contains either 3 mutual acquaintances or 6 mutual strangers.

Ramsey numbers are notoriously hard to compute. Despite decades of effort, the exact values of only a handful are known. Among these, $R(3, 6) = 18$ stands out as one of the largest Ramsey numbers for which a short, computer-free combinatorial proof is known [1]. The argument, due to Cariolaro, occupies roughly two pages and is elementary yet ingenious—making it an ideal candidate for mechanization.

Contributions

This paper describes a complete Lean 4 formalization of $R(3, 6) = 18$ with the following highlights:

1. **Complete verification.** The main theorem `ramsey_three_six : ramseyNumber 3 6 = 18` is fully proven with axiom footprint {`propext`, `Classical.choice`, `Quot.sound`}—the standard Lean 4 foundations. No `sorry`, no `native_decide`, no problem-specific axioms.

2. **The IndepSetChecker architecture.** We develop a reusable framework (Section 4) for kernel-checked combinatorial decidability on graphs. Crucially, the backtracking search is *verified*: a machine-checked completeness theorem (`hasIndepSetAux_complete`) proves that if a k-independent set exists, the search finds it. This means the `false` return value is not merely trusted by fast execution—it is an axiom-free theorem in Lean's kernel. The framework also includes a structural decomposition principle and a bridge to Mathlib's `SimpleGraph`.

3. **Formalization of $R(3, 3), R(3, 4), R(3, 5)$.** These three smaller Ramsey numbers are also fully proven and feed into the upper bound argument.

4. **Lean 4.28.0 + Mathlib4.** We build on the current stable Lean ecosystem, making the formalization immediately reproducible.

Related work

Graph-theoretic Ramsey theory has attracted formalization efforts in several systems. Greenwood and Gleason's classical proof of $R(3, 3) = 6$ and $R(3, 4) = 9$ [2] has been verified in various settings. The PBLean library [7] provides VeriPB-certificate-based pseudo-Boolean proof checking for Ramsey-type problems, with both explicit-term and reflection-based workflows. Our development is complementary: it is graph-structural, self-contained, and does not rely on external certificates or `native_decide`. To our knowledge, $R(3, 6) = 18$ has not previously been mechanized in any proof assistant.

2 Proof Overview and Lean Setup

2.1 Definitions

Our formalization builds on Mathlib's `SimpleGraph` API. The core definitions live in `RamseyDef.lean`:

```
def HasRamseyProperty (k l : Nat) (G : SimpleGraph V) [DecidableRel G.Adj
   ]
    : Prop :=
  (exists s : Finset V, G.IsNClique k s) or
  (exists s : Finset V, G.IsNIndepSet l s)

noncomputable def ramseyNumber (k l : Nat) : Nat :=
  sInf {n : Nat | n > 0 and
    forall (G : SimpleGraph (Fin n)) [DecidableRel G.Adj],
      HasRamseyProperty k l G}
```

Two abbreviations simplify later statements:

```
abbrev TriangleFree (G : SimpleGraph V) : Prop := G.CliqueFree 3
abbrev NoKIndepSet (k : Nat) (G : SimpleGraph V) : Prop := G.IndepSetFree
    k
```

A graph G *witnesses* $R(k,l) \geq n+1$ if it has n vertices, contains no k-clique, and contains no l-independent set.

2.2 The Main Theorem

```
-- MainTheorem.lean
theorem ramsey_three_six : ramseyNumber 3 6 = 18 := by
  apply Nat.le_antisymm
  . exact ramsey_three_six_upper          -- Section 5: R(3,6) <= 18
  . apply ramsey_three_six_ge_18_of_nonempty
    exact ramseySet_3_6_nonempty          -- Section 3: R(3,6) >= 18

-- Verified axiom footprint:
#print axioms ramsey_three_six
-- 'ramsey_three_six' depends on axioms:
--   [propext, Classical.choice, Quot.sound]
```

The proof splits into an upper bound (Section 5) and a lower bound (Section 3). The nonemptiness witness for the lower bound is supplied by the upper bound theorem itself.

2.3 Module Structure

Figure 1 shows the module dependency graph. Total code size: approximately 15,500 lines (of which 12,900 are the upper bound).

2.4 Reviewer Reproducibility Checklist

For ATP/ITP audit, the following checks are sufficient to validate this paper's central trust claims:

```
lake build
lake env lean Ramsey36/MainTheorem.lean
-- Expected in output:
-- 'ramsey_three_six' depends on axioms:
--   [propext, Classical.choice, Quot.sound]

rg -n "native_decide|Lean.ofReduceBool|sorry|admit" \
```

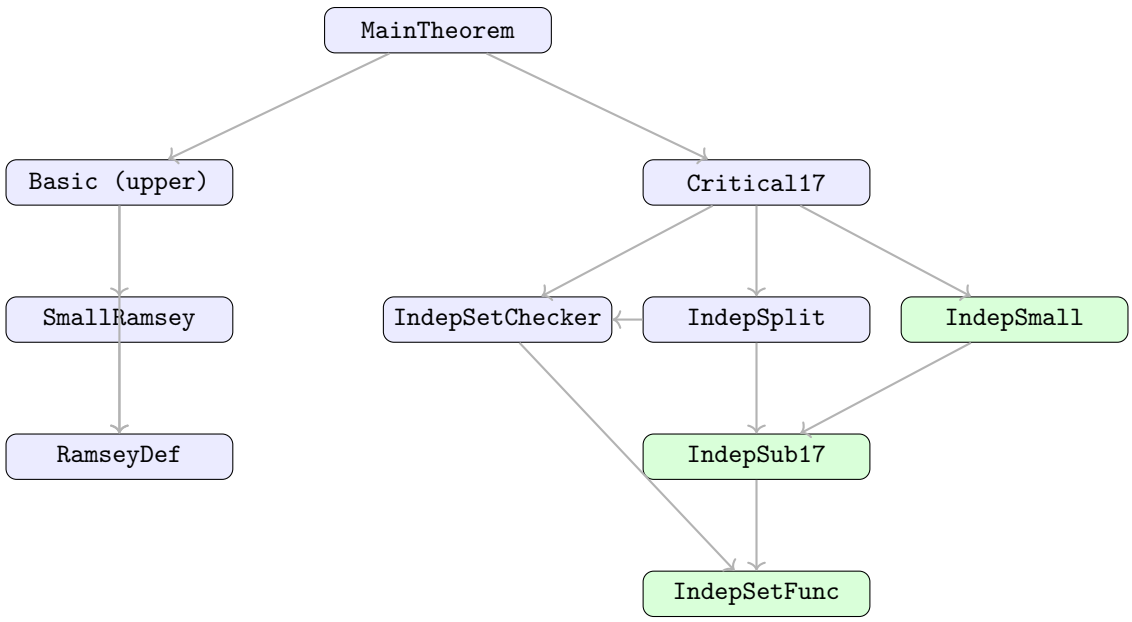

Figure 1: Module dependency graph. Green nodes (no Mathlib imports) run kernel evaluation with low memory. The blue nodes import Mathlib but are not directly involved in kernel-evaluated computations.

```
  Ramsey36 Ramsey36.lean Main.lean -g "*.lean"
-- Expected: no matches
```

3 The Lower Bound: The Graver–Yackel Graph

The lower bound $R(3,6) \geq 18$ is witnessed by the *Graver–Yackel graph*: a 17-vertex, 5-regular, triangle-free graph with independence number 5, constructed in [3].

3.1 Definition in Lean

We define the graph via an explicit neighborhood table:

```
-- Critical17.lean
def neighbors17 (v : Fin 17) : Finset (Fin 17) :=
  if v = 0 then {9, 14, 15, 16}
  else if v = 1 then {7, 11, 13, 16}
  else if v = 2 then {8, 10, 12, 15}
  -- ...
  else {0, 1, 3, 4, 10}    -- v = 16

def adj17 (v w : Fin 17) : Prop := w in neighbors17 v

def criticalGraph17 : SimpleGraph (Fin 17) where
  Adj     := adj17
  symm    := by intro v w h; exact (neighbors17_symm v w).mp h
  loopless := by
    constructor; intro v h
```

```
    unfold adj17 neighbors17 at h
    fin_cases v <;> simp at h
```

Symmetry of `neighbors17` is proved by `fin_cases v <;> fin_cases w <;> decide`, enumerating all $17 \times 17 = 289$ pairs. Each goal is a decidable finset-membership check and runs quickly.

3.2 The Lower Bound Argument

With the two key properties established (Section 4), the lower bound follows by a subgraph embedding argument: if $n \leq 17$, then the Graver–Yackel graph maps into K_n via `Fin.castLE`, and the Ramsey property pulls back through `SimpleGraph.comap`.

```
lemma not_hasRamseyProperty_17 :
    not HasRamseyProperty 3 6 criticalGraph17 := by
  unfold HasRamseyProperty; push_neg
  constructor
  . intro s h; exact criticalGraph17_triangleFree s h
  . intro s h; exact criticalGraph17_no_6_indep s h
```

4 The IndepSetChecker Architecture

The two key properties of the Graver–Yackel graph—triangle-freeness and absence of a 6-independent set—are combinatorial facts about a 17-vertex graph. This section describes our approach to verifying them in Lean's trusted kernel, without extra axioms.

4.1 The Problem with Naive Decidability

Both `TriangleFree` and `NoKIndepSet 6` are decidable via Mathlib's `Fintype` machinery. In principle, `by decide` should suffice. In practice it does not:

- **Subset enumeration.** Mathlib's `IndepSetFree` instance checks all subsets of size k. For $n = 17$, $k = 6$: $\binom{17}{6} = 12{,}376$ subsets. The kernel exhausts memory before completing the check.
- **`native_decide`.** Lean's `native_decide` compiles the check to native code and runs it outside the kernel. It succeeds, but introduces the axiom `Lean.ofReduceBool`: this enlarges the trusted computing base beyond kernel reduction. In this artifact we avoid that path and keep theorem checking kernel-native.

4.2 Four-File Architecture

Our solution is a custom backtracking search with a kernel-verifiable correctness proof. The design separates *computation* (no Mathlib) from *correctness* (with Mathlib):

File	Lines	Role
`IndepSetFunc.lean`	38	Search function; *no* Mathlib
`IndepSub17.lean`	88	12 sub-problems via `decide`; no Mathlib
`IndepSmall.lean`	93	Small Ramsey checks; no Mathlib
`IndepSplit.lean`	32	Combines sub-problems; imports Mathlib
`IndepSetChecker.lean`	222	Correctness and bridge; imports Mathlib

The critical design constraint: files that execute `by decide` computations (`IndepSub17.lean`, `IndepSmall.lean`) import *only* `IndepSetFunc.lean`. Mathlib's compiled `.olean` files occupy approximately 5 GB of virtual memory. Without them, the kernel evaluates Boolean computations in only ∼100 MB, staying well within limits.

4.3 The Search Function

The heart of the approach is `hasIndepSetAux` in `IndepSetFunc.lean`, given here in full:

```
-- IndepSetFunc.lean (complete listing)

/-- Backtracking search: returns ‘true‘ iff an independent set of size
    ‘remaining‘ exists among vertices {start, ..., n-1} extending ‘chosen
      ‘.
    Recursion depth: O(n), not O(2^n). -/
def hasIndepSetAux (n : Nat) (adj : Fin n -> Fin n -> Bool)
    (remaining : Nat) (start : Nat) (chosen : List (Fin n)) : Nat -> Bool
  | 0        => if remaining = 0 then true else false
  | fuel + 1 =>
    if remaining = 0 then true
    else if h : start < n then
      if n - start < remaining then false        -- pigeonhole prune
      else
        let v : Fin n := ⟨start, h⟩
        let compatible := chosen.all (fun w => !adj v w)
        if compatible then
          hasIndepSetAux n adj (remaining - 1) (start + 1) (v :: chosen)
             fuel ||
          hasIndepSetAux n adj remaining       (start + 1) chosen
                     fuel
        else
          hasIndepSetAux n adj remaining       (start + 1) chosen
                     fuel
    else false

def hasIndepSet (n : Nat) (adj : Fin n -> Fin n -> Bool) (k : Nat) : Bool
    :=
  hasIndepSetAux n adj k 0 [] (n + 1)
```

We explain the key design choices:

Parameters.

- `remaining`: how many more vertices still need to be selected.
- `start`: the current candidate vertex index (we only look at vertices $\geq$ `start`, enumerating each subset exactly once in lexicographic order).
- `chosen`: the list of already-selected vertices.
- `fuel`: a syntactic termination counter initialized to $n + 1$. This keeps recursion depth $O(n)$.

Pigeonhole pruning. The check `if n - start < remaining then false` terminates branches where fewer vertices remain than we still need. This embeds the pigeonhole principle directly into the search: if only m candidate vertices remain ($m <$ `remaining`), no independent set of the required size can exist.

Boolean adjacency. The search takes `adj : Fin n -> Fin n -> Bool` rather than `G.Adj : V -> V -> Prop`. Boolean functions reduce in the kernel with $O(1)$ cost per lookup (a single pattern match), whereas propositional adjacency via `Finset.mem` requires traversing a linked list. We bridge back to `SimpleGraph` after the computation (Section 4.7).

Fuel-based recursion. Lean requires structural termination arguments. Using a `Nat` fuel counter decremented at each vertex step makes the function structurally recursive. Since `start` advances by 1 per step and `start` $\leq n$, fuel $n + 1$ always suffices.

4.4 Sub-Problem Splitting

Even with the Mathlib-free design, a single `decide` on `hasIndepSet 17 adj17Bool 6` causes kernel out-of-memory. The search tree has too many nodes for a monolithic evaluation.

The solution: condition on the *first selected vertex* (vertices 0 through 11), splitting the search into 12 sub-problems. Vertices ≥ 12 cannot start a 6-independent set by pigeonhole: only $17 - 12 = 5 < 6$ vertices remain.

Each sub-problem fixes vertex i as selected and asks whether 5 more independent vertices exist among $\{i + 1, \ldots, 16\}$:

```
-- IndepSub17.lean

/-- Direct Bool adjacency for the Graver-Yackel 17-vertex graph. -/
def adj17Bool (v w : Fin 17) : Bool :=
  match v.val, w.val with
  | 0, 9  | 0, 14 | 0, 15 | 0, 16 => true
  | 1, 7  | 1, 11 | 1, 13 | 1, 16 => true
  -- ... (all 43 edges, both directions)
  | _, _ => false

set_option maxHeartbeats 800000 in
theorem sub17_0 :
    hasIndepSetAux 17 adj17Bool 5 1 [⟨0, by omega⟩] 17 = false := by
       decide

set_option maxHeartbeats 800000 in
theorem sub17_1 :
    hasIndepSetAux 17 adj17Bool 5 2 [⟨1, by omega⟩] 16 = false := by
       decide

-- ... sub17_2 through sub17_10 ...

set_option maxHeartbeats 200000 in
theorem sub17_11 :
    hasIndepSetAux 17 adj17Bool 5 12 [⟨11, by omega⟩] 6 = false := by
       decide
```

Each sub-problem has at most ∼970 search nodes and verifies in under one second. Heartbeat limits decrease for later vertices (reflecting the shrinking search space).

Defining adjacency as an exhaustive `match` rather than via `Finset.mem` avoids linked-list traversal overhead during kernel evaluation.

4.5 Structural Decomposition: Combining Sub-Problems

The 12 sub-problems are combined in `IndepSplit.lean` using the structural decomposition lemma from `IndepSetChecker.lean`:

```
-- IndepSetChecker.lean

/-- If both the include-v and skip-v branches return false, the full
    call returns false.  Used to chain sub-problem results. -/
theorem hasIndepSetAux_false_of_compat
    {n : Nat} {adj : Fin n -> Fin n -> Bool}
    {remaining start fuel : Nat} {chosen : List (Fin n)}
    (h_rem    : remaining /= 0)
    (h_start  : start < n)
    (h_pig    : not (n - start < remaining))
    (h_compat : (chosen.all fun w => !adj ⟨start, h_start⟩ w) = true)
    (h_inc  : hasIndepSetAux n adj (remaining-1) (start+1)
                (⟨start, h_start⟩ :: chosen) fuel = false)
    (h_skip : hasIndepSetAux n adj remaining (start+1) chosen fuel =
       false) :
    hasIndepSetAux n adj remaining start chosen (fuel + 1) = false
```

The combination proof chains 12 applications of this lemma:

```
-- IndepSplit.lean

theorem hasIndepSet_17_adj17Bool_6_false :
    hasIndepSet 17 adj17Bool 6 = false := by
  show hasIndepSetAux 17 adj17Bool 6 0 [] 18 = false
  exact hasIndepSetAux_false_of_compat (...) sub17_0 <|
  hasIndepSetAux_false_of_compat (...) sub17_1 <|
  -- ...
  hasIndepSetAux_false_of_compat (...) sub17_11 <|
  hasIndepSetAux_false_of_pig (by decide) (by omega) (by omega)
```

The final `hasIndepSetAux_false_of_pig` handles the pigeonhole base case: no 6-IS can start with a vertex ≥ 12.

4.6 Correctness: The Completeness Theorem

The search function is *sound* by construction (when it returns `true`, `chosen` is a valid independent set by invariant of the loop—each vertex is added only when compatible with all prior choices). This direction is *not* stated as a separate theorem because all applications in this work only use `false` results. The non-trivial and mechanically verified direction is *completeness*: if a k-independent set exists, the search returns `true`. This is the theorem that makes the entire architecture formally sound: a `false` result is not merely trusted by fast execution—it is an axiom-free kernel theorem whose proof term Lean's type-checker certifies. Without this theorem, the checker would be an unverified oracle.

```
-- IndepSetChecker.lean

/-- If a k-independent set exists, the backtracking search returns true.
    -/
theorem hasIndepSetAux_complete
    {n : Nat} {adj : Fin n -> Fin n -> Bool}
    {remaining start fuel : Nat} {chosen : List (Fin n)}
    {s : Finset (Fin n)}
    (h_fuel   : n <= start + fuel)
    (h_card   : s.card = remaining)
    (h_ge     : forall v in s, start <= v.val)
    (h_indep  : forall v in s, forall w in s,
                  v /= w -> adj v w = false)
    (h_compat : forall v in s, forall w in chosen,
                  adj v w = false) :
    hasIndepSetAux n adj remaining start chosen fuel = true
```

Proof sketch. By induction on `fuel`. **Base case** (`fuel = 0`): if any vertex of s remained, it would need index $\geq$ `start` $\geq n$ (by `h_fuel`), contradicting the type bound on `Fin n`. Hence s is empty, so `remaining = 0`, and the function returns `true`.

Inductive step: the pigeonhole guard passes since $|s| =$ `remaining` elements all have index $\geq$ `start`. Case-split on whether `start` $\in s$:

- If yes: `h_compat` ensures the compatibility check passes. Apply the inductive hypothesis to $s \setminus \{$`start`$\}$ to obtain `true` for the include branch.
- If no: all elements of s have index $\geq$ `start` $+ 1$. Apply the inductive hypothesis to s for the skip branch.

□

The contrapositive is what we need: if `hasIndepSetAux ... = false`, then no k-independent set of the required form exists.

4.7 Bridge to Mathlib's `SimpleGraph`

The final step connects the Boolean computation to Mathlib's propositional `NoKIndepSet` predicate. We generalize the bridge to accept any Boolean adjacency function that agrees pointwise with the graph's adjacency relation:

```
-- IndepSetChecker.lean

/-- If `adj` agrees with `decide (G.Adj . .)` pointwise,
    and the checker returns false, then G has no k-independent set. -/
theorem noKIndepSet_of_adj_checker_false
    {n k : Nat} {G : SimpleGraph (Fin n)} [DecidableRel G.Adj]
    {adj : Fin n -> Fin n -> Bool}
    (hadj : forall v w : Fin n, adj v w = decide (G.Adj v w))
    (h    : hasIndepSet n adj k = false) :
    NoKIndepSet k G := by
  apply noKIndepSet_of_checker_false
  have heq : adj = fun v w => decide (G.Adj v w) :=
```

```
    funext (fun v => funext (fun w => hadj v w))
  rwa [<- heq]
```

In `Critical17.lean`, we instantiate this with a `decide` proof covering all $17 \times 17 = 289$ pairs:

```
-- Critical17.lean

/-- adj17Bool agrees with criticalGraph17.Adj on all 289 pairs.
    Each pair costs O(5) kernel steps (degree-5 finset lookup). -/
private lemma adj17Bool_spec :
    forall v w : Fin 17,
      adj17Bool v w = decide (criticalGraph17.Adj v w) := by decide

lemma criticalGraph17_no_6_indep : NoKIndepSet 6 criticalGraph17 :=
  noKIndepSet_of_adj_checker_false adj17Bool_spec
    hasIndepSet_17_adj17Bool_6_false
```

The `decide` call in `adj17Bool_spec` performs $17 \times 17 = 289$ finset membership lookups, each of cost $O(5)$ (the graph is 5-regular), for $\sim$1,500 total kernel steps—well within limits.

Remark 4.1 (A crucial pitfall with `decide` and free variables)**.** *Lean's* `decide` *tactic requires a* closed *proposition (no free variables). Writing*

```
-- WRONG: v and w are free variables in the local context
private lemma adj17Bool_spec (v w : Fin 17) :
    adj17Bool v w = decide (criticalGraph17.Adj v w) := by decide
```

fails with `Expected type must not contain free variables`*. The fix is to lift the universal quantifier into the proposition itself:*

```
-- CORRECT: closed proposition, decide works
private lemma adj17Bool_spec :
    forall v w : Fin 17,
      adj17Bool v w = decide (criticalGraph17.Adj v w) := by decide
```

4.8 Triangle-Freeness via the Complement

A triangle in G is a 3-clique, which corresponds to a 3-independent-set in the *complement* graph. We exploit this duality to reuse the same checker infrastructure:

```
-- IndepSmall.lean

/-- Complement adjacency: edge iff NOT adjacent in the original graph.
    A 3-IS in adj17NotBool = a 3-clique in adj17Bool = a triangle. -/
def adj17NotBool (v w : Fin 17) : Bool := !adj17Bool v w

-- Single fast check: the complement has 93 edges, 3-clique absent.
set_option maxHeartbeats 400000 in
theorem hasIndepSet_17_adj17NotBool_3_false :
    hasIndepSet 17 adj17NotBool 3 = false := by decide
```

The corresponding bridge theorem and instantiation:

```
-- IndepSetChecker.lean
```

```
theorem triangleFree_of_adj_checker_false
    {n : Nat} {G : SimpleGraph (Fin n)} [DecidableRel G.Adj]
    {adjNot : Fin n -> Fin n -> Bool}
    (hadj : forall v w : Fin n, adjNot v w = !decide (G.Adj v w))
    (h : hasIndepSet n adjNot 3 = false) : TriangleFree G

-- Critical17.lean

private lemma adj17NotBool_spec :
    forall v w : Fin 17,
      adj17NotBool v w = !decide (criticalGraph17.Adj v w) := by decide

lemma criticalGraph17_triangleFree : TriangleFree criticalGraph17 :=
  triangleFree_of_adj_checker_false adj17NotBool_spec
    hasIndepSet_17_adj17NotBool_3_false
```

4.9 Summary of the Architecture

The IndepSetChecker achieves kernel-verified combinatorial decidability through four interlocking principles:

1. **Separation of computation and verification.** The search function (no Mathlib, $\sim$100 MB) and correctness proof (with Mathlib) live in separate files.
2. **Fuel-bounded backtracking.** Linear recursion depth avoids the $O(2^n)$ stack blow-up of naive enumeration.
3. **Structural decomposition.** Large searches are split into small sub-problems, each independently verifiable by `decide`, then combined via a chain of bridge lemmas.
4. **Complement duality.** Triangle-freeness reuses the independent-set checker on the complement, requiring no separate infrastructure.

5 The Upper Bound: Formalizing Cariolaro's Proof

The upper bound $R(3,6) \leq 18$ is the bulk of the formalization, occupying `Basic.lean` ($\sim$12,900 lines). We outline the three main claims from Cariolaro's proof [1] and note key Lean-specific aspects.

5.1 Setup

Throughout, G is a triangle-free graph on 18 vertices with no 6-independent set. We derive a contradiction.

A foundational observation from `RamseyDef.lean`:

```
/-- In a triangle-free graph, the neighborhood of any vertex is
    independent. -/
lemma neighborSet_indep_of_triangleFree
    {G : SimpleGraph V} (h : TriangleFree G) (v : V) :
    G.IsIndepSet (G.neighborSet v)
```

Since $N(v)$ is independent and G has no 6-IS, we immediately get $\deg(v) \leq 5$ for all v.

5.2 Claim 1: G is 5-Regular

Lemma 5.1 (Formalized as `claim1_five_regular`). *G is 5-regular.*

Upper bound on degree. $N(v)$ is independent with $|N(v)| = \deg(v)$, so $\deg(v) \leq 5$.
Lower bound on degree. Suppose $\deg(v) < 5$. Let $H = G - N[v]$; then $|H| \geq 13$.

- If $|H| \geq 14 = R(3,5)$: since $R(3,5) = 14$ is formalized (Section 6), H contains a 5-IS, which together with v gives a 6-IS—contradiction.
- If $|H| = 13$ (so $\deg(v) = 4$): H must be the unique $R(3,5)$-critical graph (Paley(13), 4-regular). A counting argument via a neighbor $t \in N(v)$ constructs a 6-IS from $(N(v) \setminus \{t\}) \cup \{t_1, t_2, t_3\}$ where $t_i \in H$ are neighbors of t.

This argument relies directly on the previously formalized $R(3,5) = 14$, illustrating how the small Ramsey results feed into the main proof.

5.3 Claim 2: Neighbor Structure

Lemma 5.2 (Formalized as `claim2_neighbor_structure`). *For any non-adjacent pair u, v: $1 \leq |N(u) \cap N(v)| \leq 2$. Moreover, for any vertex v, exactly 4 non-neighbors share exactly 1 common neighbor with v (the set P), and 8 non-neighbors share exactly 2 (the set $Q \cup W$).*

- **Lower bound.** If $N(u) \cap N(v) = \emptyset$, then $\{u\} \cup N(u)$ is a 6-IS.
- **Upper bound.** If $|N(u) \cap N(v)| \geq 3$, then $G - N[u,v]$ has $\geq 9 = R(3,4)$ vertices, containing a 4-IS; together with u, v this gives a 6-IS.

The counting uses `commonNeighbors` from `RamseyDef.lean` and a double-counting argument on edges between $G - N[v]$ and $N[v]$.

5.4 Claim 3: P Induces a 4-Cycle

Lemma 5.3 (Formalized as `claim3_four_cycle`). *The set P of 4 non-neighbors of v that each share exactly one common neighbor with v induces a 4-cycle in G.*

The key tool is the **FiveCycleLemma** (`FiveCycleLemma.lean`):

Theorem 5.4 (Five-cycle structure). *In a 5-vertex triangle-free graph with no 3-independent set, every vertex has degree exactly 2 (i.e., the graph is the 5-cycle C_5).*

Proof sketch. Min degree ≥ 2. If $\deg(v) \leq 1$, the 3 non-neighbors of v contain at most 1 edge (triangle-free), leaving a 3-IS with v. *Max degree ≤ 2.* If $\deg(v) \geq 3$, three neighbors form a 3-IS since $N(v)$ is independent (triangle-free). □

The lemma is applied to the 5-vertex induced subgraph on $\{v\} \cup P$: each $p_i \in P$ has exactly one neighbor in $N(v)$ (by definition of P), so the p_i's have degree 2 in this subgraph—forcing the 4-cycle structure.

5.5 Final Contradiction

Cariolaro's labeling: $V(G) = \{v, t, s_1, s_2, s_3, s_4, t_1, t_2, t_3, t_4, p_1, p_2, p_3, p_4, w_1, w_2, w_3, w_4\}$ where $N(v) = \{t, s_1, s_2, s_3, s_4\}$, $N(t) \setminus \{v\} = \{t_1, t_2, t_3, t_4\}$, $P = \{p_1, p_2, p_3, p_4\}$ induces the 4-cycle $p_1p_2p_3p_4p_1$, and $W = \{w_1, w_2, w_3, w_4\}$.

A careful case analysis (formalized as `final_contradiction` in `Basic.lean`) shows: w_1 is adjacent to s_3 and t_1, and the edge $w_1s_2 \in E(G)$ forces $s_2t_1 \in E(G)$. But then s_2 and p_1 share three common neighbors $\{p_2, w_1, t_1\}$, contradicting Claim 2. □

The main upper bound theorem is then:

```
theorem hasRamseyProperty_3_6_18 :
    0 < 18 and forall (G : SimpleGraph (Fin 18)) [DecidableRel G.Adj],
      HasRamseyProperty 3 6 G := by
  constructor
  . simp
  . intro G inst
    by_contra h_not_ramsey
    unfold HasRamseyProperty at h_not_ramsey; push_neg at h_not_ramsey
    rcases h_not_ramsey with ⟨h_no_clique, h_no_indep⟩
    have h_tri : TriangleFree G      := fun t ht => h_no_clique t ht
    have h_no6 : NoKIndepSet 6 G     := fun t ht => h_no_indep  t ht
    have h_reg : IsKRegular G 5      := claim1_five_regular h_tri h_no6
    exact final_contradiction h_reg h_tri h_no6
```

6 Small Ramsey Numbers as Dependencies

The upper bound for $R(3,6) = 18$ (specifically, Claim 1) relies on the values of $R(3,3) = 6$, $R(3,4) = 9$, and $R(3,5) = 14$. All three are fully formalized in `SmallRamsey.lean` using the same IndepSetChecker architecture, with computations in `IndepSmall.lean`.

Theorem	Critical graph	n	Computation
$R(3,3) = 6$	C_5 (5-cycle)	5	`hasIndepSet 5 adj5Bool 3 = false`
$R(3,4) = 9$	$C_8(1,4)$	8	`hasIndepSet 8 adj8Bool 4 = false`
$R(3,5) = 14$	Paley(13)	13	`hasIndepSet 13 adj13Bool 5 = false`

For example, the Paley(13) computation (quadratic-residue adjacency mod 13):

```
-- IndepSmall.lean

def adj13Bool (v w : Fin 13) : Bool :=
  match v.val, w.val with
  | 0, 1  | 0, 5  | 0, 8  | 0, 12 => true
  | 1, 0  | 1, 2  | 1, 6  | 1,  9 => true
  -- ...
  | _, _ => false

set_option maxHeartbeats 400000 in
theorem hasIndepSet_13_adj13Bool_5_false :
    hasIndepSet 13 adj13Bool 5 = false := by decide
```

The structure—explicit adjacency table, no Mathlib, `by decide`—is identical to the 17-vertex case, but fits in a single `decide` call since $\binom{13}{5}$ is small enough.

7 Equivalence with the Textbook Definition

A potential objection to the `sInf`-based definition of `ramseyNumber` is that it might not match the standard combinatorial definition in the literature. The file `RamseyAlt.lean` closes this gap formally.

7.1 The Textbook Definition

Following Radziszowski's survey [4], $R(k, l)$ is the *least positive integer* n such that every graph on n vertices contains either a k-clique or an l-independent set. Translated directly into Lean:

```
-- RamseyAlt.lean

/-- Classical characterization (Radziszowski 1994):
    R(k,l) is the LEAST positive n such that
    every n-vertex graph has a k-clique or l-IS.
    (i)  n > 0
    (ii) every n-vertex graph has the Ramsey property
    (iii) for each 0 < m < n, some m-vertex graph lacks it -/
def IsRamseyNumber (k l n : Nat) : Prop :=
  0 < n and
  (forall (G : SimpleGraph (Fin n)) [DecidableRel G.Adj],
    HasRamseyProperty k l G) and
  (forall m : Nat, 0 < m -> m < n ->
    exists (G : SimpleGraph (Fin m))
           (_ : DecidableRel G.Adj),
      not HasRamseyProperty k l G)
```

Condition (iii) is the critical one: it certifies that n is truly minimal.

7.2 Instance Independence

`HasRamseyProperty k l G` is defined as $(\exists s,\ G.\text{IsNClique}\ k\ s) \vee (\exists s,\ G.\text{IsNIndepSet}\ l\ s)$, which does not reference the `DecidableRel G.Adj` instance. Therefore the proposition is instance-independent:

```
lemma hasRamseyProperty_instIndep {V : Type*}
    (k l : Nat) (G : SimpleGraph V)
    (inst1 inst2 : DecidableRel G.Adj) :
    @HasRamseyProperty V k l G inst1 <->
    @HasRamseyProperty V k l G inst2 := Iff.rfl
```

This means the set over which `sInf` is taken in `ramseyNumber` is unaffected by instance choices.

7.3 Equivalence Theorem

Theorem 7.1 (Formalized as `isRamseyNumber_iff_eq_ramseyNumber`)**.** *When the Ramsey set is nonempty,* `IsRamseyNumber` $k\ l\ n \iff n =$ `ramseyNumber` $k\ l$.

Proof sketch. Forward. If n satisfies conditions (i)–(iii), then n belongs to the Ramsey set, so `sInf` $\leq n$. If `ramseyNumber` $< n$, condition (iii) produces a graph on `ramseyNumber` vertices lacking the Ramsey property, contradicting that `ramseyNumber` is *in* the Ramsey set (condition (ii)). Hence $n =$ `ramseyNumber`.

Backward. `sInf` of a nonempty subset of $\mathbb{N}$ belongs to the set (`Nat.sInf_mem`) and is a lower bound (`Nat.sInf_le`). These immediately give conditions (i)–(iii). □

7.4 Main Instantiation

```
-- RamseyAlt.lean

theorem isRamseyNumber_3_6_18 : IsRamseyNumber 3 6 18 :=
  (isRamseyNumber_iff_eq_ramseyNumber
      ramseySet_3_6_nonempty').mpr
    ramsey_three_six.symm
```

Verified axiom footprint:

```
#print axioms isRamseyNumber_3_6_18
-- 'isRamseyNumber_3_6_18' depends on axioms:
--   [propext, Classical.choice, Quot.sound]
```

This confirms that `ramseyNumber 3 6` in our formalization is exactly the classical combinatorial object defined in the literature.

8 Axiom Footprint and Trust

```
#print axioms ramsey_three_six
-- 'ramsey_three_six' depends on axioms:
--   [propext, Classical.choice, Quot.sound]
```

The three axioms are the standard Lean 4 foundations:

- `propext`: logically equivalent propositions are equal.
- `Classical.choice`: non-constructive choice, needed for `sInf` in the definition of `ramseyNumber`.
- `Quot.sound`: quotient soundness, used throughout Mathlib's algebra and graph theory infrastructure.

Notably absent:

- `Lean.ofReduceBool`: the axiom introduced by `native_decide`. Our use of `by decide` with Mathlib-free sub-problem files means all computations are verified by Lean's trusted kernel.
- `sorry` / **custom axioms**: none.

Trust discussion. Lean's kernel is a small, independently auditable piece of code (the *trusted computing base*). A proof accepted by the kernel with the above three axioms carries the same epistemic status as a traditional mathematical proof in ZFC with propositional extensionality. By contrast, `native_decide` delegates verification to a compiled binary and therefore depends on additional compiler/runtime trust assumptions. The IndepSetChecker architecture is specifically designed to avoid this trust gap.

9 Conclusion

We have presented a complete Lean 4 mechanization of $R(3, 6) = 18$, with axiom footprint confined to the three standard Lean foundations.

The key technical contribution is the *IndepSetChecker*: a reusable, kernel-efficient framework for combinatorial decidability on graphs, combining:

1. a fuel-bounded backtracking search with pigeonhole pruning (Mathlib-free),
2. a completeness theorem (with Mathlib),
3. a structural decomposition principle for splitting large computations, and
4. a clean pointwise bridge from Boolean adjacency to Mathlib's `SimpleGraph`.

This architecture addresses a genuine tension in interactive theorem proving: combinatorial properties of small graphs are in principle decidable, but naive kernel evaluation is infeasible and `native_decide` expands the trusted base. We thread this needle by separating computation from correctness, keeping evaluation Mathlib-free, and splitting large computations into kernel-digestible sub-problems.

Future work. The next natural targets are $R(3,7) = 23$ and $R(4,4) = 18$. The IndepSetChecker scales: $R(3,7)$ would require $\sim$100–200 sub-problems for the 22-vertex critical graph. Integration with PBLean [7] via VeriPB certificates is another avenue, providing an independent computational witness.

Acknowledgments

The authors thank **Lawrence Paulson** (University of Cambridge) for the key suggestion to replace `native_decide` with a verified backtracking algorithm whose correctness is certified by a completeness theorem inside the kernel—the design principle at the heart of the IndepSetChecker architecture. We thank **Josef Urban** (Czech Technical University in Prague) for proposing $R(3,6) = 18$ as a "proof-of-capacity" autoformalization benchmark and for encouraging a machine-verifiable account of the full formalization pipeline.

References

[1] D. Cariolaro. On the Ramsey number $R(3,6)$. *Research Report R-99-2012*, Aalborg University, Denmark, 1999.

[2] R. E. Greenwood and A. M. Gleason. Combinatorial relations and chromatic graphs. *Canadian Journal of Mathematics*, 7:1–7, 1955.

[3] J. E. Graver and J. Yackel. Some graph theoretic results associated with Ramsey's theorem. *Journal of Combinatorial Theory*, 4:125–175, 1968.

[4] S. P. Radziszowski. Small Ramsey numbers. *Electronic Journal of Combinatorics*, Dynamic Survey DS1, 1994 (updated regularly).

[5] L. de Moura, S. Ullrich, et al. The Lean 4 theorem prover and programming language. In *Proceedings of CADE-28*, 2021.

[6] The Mathlib Community. Mathlib: A unified library of mathematics for Lean 4. `https://leanprover-community.github.io/`, 2024.

[7] J. Blanchette, H. Manthey, S. Szeider, et al. PBLean: Pseudo-Boolean proof checking in Lean 4. Preprint, 2024–2026.